

TRIS v1.3

Web Developer Scope of Work & Build Specification

Trust & Risk Intelligence System (TRIS)

Current stage	Existing web-based demonstration prototype; next version is v1.3
Primary goal	Convert one core TRIS workflow into a functional, data-driven, testable web application using synthetic data.
Development approach	Preserve the current version, build incrementally, document changes, and validate each feature.
Data rule	Synthetic data only for this phase. No confidential employer/customer/supplier data.

IMPORTANT: Do not rebuild TRIS from scratch unless a technical assessment proves the existing codebase cannot reasonably be extended.

Prepared for developer handoff - August 31, 2026

1. Project Objective

TRIS v1.3 should demonstrate one complete risk-intelligence workflow, not every possible TRIS module.

The immediate objective is to make one financial/supplier-risk scenario function end-to-end with stored data, transparent rules, case ownership, corrective action, verified closure, and recurrence tracking.

Core workflow: Ingest -> Validate -> Establish Baseline -> Detect -> Connect -> Explain -> Prioritize -> Assign Owner -> Investigate -> Root Cause -> Corrective Action -> Verify Closure -> Monitor Recurrence.

2. Existing System: What Must Be Preserved

- Review the existing TRIS source code and identify the current framework, packages, deployment method, hosting, and whether any backend/database already exists.
- Preserve the current working prototype as the historical/baseline version (v1.2 or equivalent tag/branch).
- Create a separate development branch/environment for v1.3. Do not overwrite the historical version.
- Keep the project under the project owner's GitHub/Vercel/database accounts where possible. The developer should be added as a collaborator rather than becoming the sole owner of the project.
- Provide a staging/development URL before any v1.3 changes are promoted to the primary public demonstration URL.

3. Milestone 1 - Technical Assessment (No Code Changes Yet)

Before development begins, provide a short written technical assessment answering the following:

1. What framework and language does the current TRIS web application use?
2. Where is the source code stored and how is it versioned?
3. How is the current application deployed to Vercel?
4. Is there already a backend, API layer, authentication, or database?
5. Which existing screens/components can be reused for v1.3?
6. What technical debt or security issues should be addressed before adding data-driven features?
7. What backend/database architecture do you recommend for v1.3 and why?
8. What is the proposed milestone plan, estimate, and implementation sequence?

Developer should not begin a full rebuild during Milestone 1. The purpose is to understand and protect the existing project first.

4. Required v1.3 Development Scope

A. Data layer: Create a database and backend/API layer capable of storing synthetic suppliers, transactions, approvals, access events, risk cases, case history, root causes, corrective actions, closure evidence, and recurrence references.

B. Data ingestion: Allow the development environment to load or import the supplied synthetic dataset. Validate required fields, types, missing values, duplicate IDs, and invalid relationships.

C. Baseline analysis: For the selected supplier, calculate and display a transparent historical baseline (for example average, median, count, range, and current deviation). The baseline should exclude the transaction currently being evaluated.

D. Rules-based detection: Implement configurable demonstration rules such as amount deviation, recent supplier bank-detail change, missing required approval, unusual access time, duplicate invoice, and recurrence.

E. Explainability: Every triggered rule must produce a plain-language reason. Do not display unsupported fraud probabilities, fake AI confidence, or unvalidated accuracy percentages.

F. Case generation: Related signals for one event should be grouped into a single case when the configured logic indicates they belong together.

G. Priority: Assign High/Medium/Low priority using transparent, configurable demonstration logic. Priority rules must be visible to the project owner and easy to change.

H. Ownership & workflow: Allow a case to be assigned to an owner/department and moved through New -> Assigned -> Under Investigation -> Corrective Action -> Pending Verification -> Closed.

I. Root cause & corrective action: Provide structured fields for investigation notes, root-cause category/description, corrective action, responsible party, due date, and completion status.

J. Verified closure: Closing a case must require defined closure information such as closure type, corrective action, supporting evidence/reference, verifier, verification date, and follow-up/recurrence monitoring status.

K. Recurrence: When a similar later issue occurs, show relevant prior cases, prior root cause, corrective action, closure, and whether the issue appears to have recurred.

L. Audit trail: Preserve timestamped case history for owner changes, status changes, notes, root cause, corrective action, verification, and closure.

5. Primary End-to-End Test Scenario

The supplied spreadsheet contains a synthetic supplier, Northstar Components LLC (SUP-001), with normal historical invoices around the \$30,000 range. A later transaction (TX-1999) is intentionally designed to trigger the primary TRIS workflow.

Signal	Synthetic condition	Expected TRIS behavior
Amount anomaly	\$104,000 transaction after a history of roughly \$28,500-\$32,100 invoices	Compare against historical baseline and explain the deviation.
Recent bank change	Supplier bank details changed on 2026-08-26	Flag as recent within the configurable demo window.
Missing approval	TX-1999 requires approval but approval status is Missing	Show required approval as a separate risk reason.
Unusual access	Related supplier activity occurs at 22:47	Link the relevant access event and explain the unusual-time rule.
Combined case	Multiple related signals exist for the same supplier/transaction context	Generate one High-priority synthetic test case with all reasons visible.

Expected case ID: TEST-CASE-001. This case is a controlled development test. It must not be described as confirmed fraud or as a validated probability of fraud.

6. Minimum Screens / Functions for v1.3

- Data Import / Data Preview - show successful rows, rejected rows, validation issues, and dataset version.
- Baseline & Pattern Analysis - show supplier history, count, average/median/range, current value, and deviation in plain language.
- Risk Alerts / Exceptions - list generated exceptions by priority, supplier, transaction, reasons, owner, and status.

- Case Detail - show all linked signals, explanations, history, ownership, investigation, root cause, corrective action, closure, and recurrence.
- Case Management - assign owner/department, due date, status, and notes.
- Verified Closure - require evidence/reference and verification before a case can be marked Closed.
- Recurrence / Prior Case View - show relevant earlier cases and previous corrective actions when a similar issue appears.
- Developer/Test View or Log - make it possible to see which rule versions ran against which data version.

7. Minimum Data Model

Entity	Minimum purpose	Example fields
Suppliers	Supplier context and change history	supplier_id, name, category, risk tier, bank_change_date
Transactions	Financial events for baseline and exception testing	transaction_id, supplier_id, invoice_no, dates, amount, approval status, payment status
Approvals	Required control evidence	approval_id, transaction_id, required level, status, approver role/date
Access Events	Contextual access/security signals	event_id, user, timestamp, system, action, resource, supplier link
Risk Cases	One consolidated exception/case record	case_id, reasons, priority, owner, status, root cause, corrective action, closure fields
Case History	Immutable/append-only history of important case changes	case_id, timestamp, actor, action, before/after status, note

8. Demonstration Rule Set

The spreadsheet contains the initial demonstration rules. These are starting test conditions, not validated production thresholds. They should be configurable rather than hard-coded into the interface.

Rule	Demo condition	Required output
R-001 Amount deviation	Current amount > 2.0x supplier historical average	Explain actual amount, baseline, and deviation.
R-002 Recent bank change	Bank details changed within previous 7 days	Explain the date and configured lookback window.
R-003 Missing approval	Approval required and not Approved	Explain which approval is missing.
R-004 Unusual access	Relevant access outside configured 06:00-20:00 window	Show timestamp and why it is outside the configured window.
R-005 Duplicate invoice	Same supplier_id + invoice_no combination exists more than once	Reference the earlier matching transaction.

Rule	Demo condition	Required output
R-006 Recurrence	Similar supplier/root-cause issue within configured 90-day window	Surface prior case and prior corrective action.

9. Verified Closure Requirements

A case should not be considered closed merely because a user clicks a button. The system should require the following before transition to Closed:

- Root cause documented.
- Corrective action documented.
- Closure type selected (for example Financial/Control, Supplier, Access/Security, Compliance, or Process).
- Supporting evidence or reference recorded.
- Verifier identified.
- Verification date recorded.
- Follow-up/retest requirement recorded.
- Recurrence-monitoring status recorded.

10. Testing & Acceptance Criteria

The supplied workbook includes a Developer_Tests sheet. At minimum, v1.3 should be considered ready for review when the following are demonstrated and evidence is preserved:

- Synthetic data imports successfully and invalid rows are handled clearly.
- SUP-001 baseline is calculated from historical data and excludes TX-1999 from its own baseline.
- TX-1999 triggers the expected linked rule reasons and creates one explainable High-priority test case.
- Normal transactions do not create unnecessary material cases under the demo rules.
- Duplicate invoice test identifies and references the earlier matching record.
- Owner assignment and status changes persist after refresh/reload.
- Case closure is blocked until required verification information is supplied.
- Case history records meaningful changes with timestamps.
- A later similar test event can surface the prior case as possible recurrence.
- No unsupported accuracy, fraud-probability, correlation, or AI-performance claims are shown.

11. Technical Expectations

Use the existing stack where practical. If changes are recommended, explain the tradeoff before implementation. Preferred capabilities include:

- Modern web stack compatible with the existing project (for example React/Next.js if that is already in use).
- Backend/API layer using a maintainable server-side approach such as Node.js/TypeScript or Python, depending on the existing codebase.
- PostgreSQL/Supabase or another documented relational database suitable for the prototype.
- GitHub version control with clear branches/tags and meaningful commits.
- Vercel-compatible staging and deployment process.
- Environment variables/secrets must not be hard-coded into source code.
- Basic role-aware access in the staging prototype if authentication is already present; otherwise document it as a later security milestone.

12. Data, Security & Confidentiality Boundaries

USE SYNTHETIC DATA ONLY IN THIS PHASE.

- Do not upload or copy confidential employer, customer, supplier, banking, payroll, production, or personally identifiable data into the public demonstration environment.
- Do not connect to an employer ERP, data warehouse, bank, MES, access-control system, or other real production source without documented authorization and a separate secure integration plan.
- Any future enterprise connector should be designed as authorized, least-privilege, and preferably read-only where appropriate for risk analysis.
- Do not market or label v1.3 as production-ready, certified, fully Zero Trust, or validated AI/ML unless later evidence independently supports those claims.

13. Out of Scope for v1.3

- Mobile application development.
- Production ERP/SAP/Business Central/Oracle integration.
- Live employer data.
- Machine-learning model training or claims of AI detection accuracy.
- Full Zero Trust architecture/certification.
- Every TRIS module or industry use case.
- Automated financial decisions without human review.
- Claims of proven savings, fraud prevention, or production performance.

14. Required Developer Deliverables

#	Deliverable	Acceptance expectation
1	Technical assessment	Short architecture/stack review before code changes.
2	Protected historical version	Existing TRIS preserved/tagged/branched before v1.3 work.
3	v1.3 staging environment	Separate working URL for review/testing.
4	Source code	All source code and configuration committed to the project owner's repository.
5	Database schema/migrations	Documented schema and any migration/setup instructions.
6	Working end-to-end test case	TEST-CASE-001 fully demonstrated from data through verified closure.
7	Testing evidence	Results for the Developer_Tests sheet with screenshots/logs and pass/fail status.
8	Change log	What changed from the preserved version to v1.3 and why.

#	Deliverable	Acceptance expectation
9	Handover notes	How to run locally, configure environment, deploy, load synthetic data, and modify rules.

15. Files Supplied With This Scope

- TRIS_v1_3_Web_Developer_Scope_of_Work.docx - this development specification.
- TRIS_v1_3_Synthetic_Test_Data.xlsx - synthetic suppliers, transactions, access events, approvals, demo rules, expected cases, sample workflow, developer tests, and data dictionary.

Developer acknowledgement: Before work begins, confirm that the current TRIS version will be preserved, the supplied data is synthetic, and v1.3 will be built incrementally against the acceptance criteria in this document.